Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

高级面向对象编程

张行健

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Development Environment & Java Datatype

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

아주대학교
AJOU UNIVERSITY

开发环境 & Java 数据类型

讲师：张行杰

邮箱：xzhang@ajou.ac.kr

微信：runningSnaile

CONTENTS

01 JAVA Development Environment

02 Buildtime and Runtime

03 JAVA vs. C/C++

04 JAVA Datatype

아주대학교 AJOU UNIVERSITY

目录

01 JAVA 开发环境

02 构建时和运行时

03 JAVA vs. C/C++

04 JAVA 数据类型

아주대학교 AJOU UNIVERSITY

01

JAVA Development Environment

01

JAVA 开发环境

아주대학교 AJOU UNIVERSITY

JDK & JRE

- Java Development Kit (JDK)
  - Includes **JRE**
  - Includes program development tools such as compilers
  - Version is important (matches language version)
  - Complies with **Java SE(Standard Edition)** specification

- Java Runtime Environment (JRE)
  - Packages required to run Java programs
  - JVM, various commands
  - Class libraries

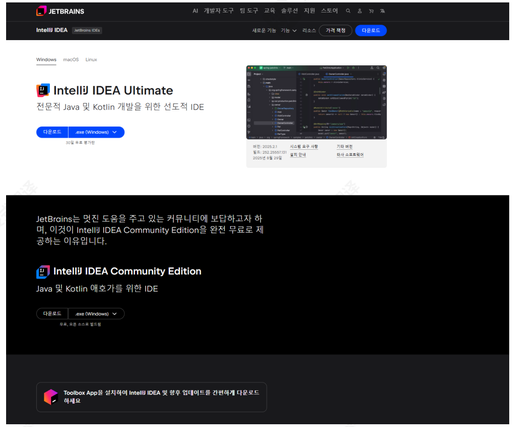
JDK & JRE

- Java Development Kit (JDK)
  - 包含JRE
  - 包含编译器等程序开发工具
  - 版本很重要（与语言版本匹配）
  - 符合Java SE(标准版)规范

- Java运行时环境 (JRE)
  - 运行Java程序所需的软件包
  - JVM，各种命令
  - 类库

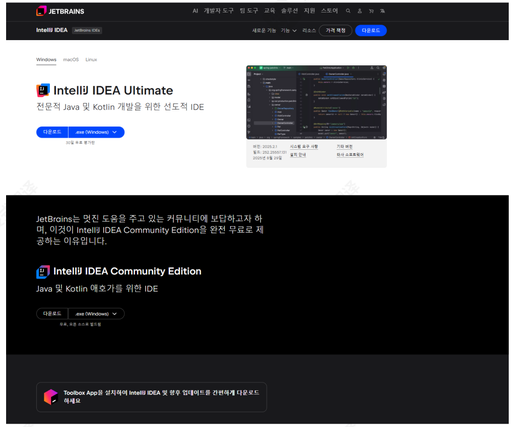
IntelliJ Download

- IntelliJ
  - https://www.jetbrains.com/ko-kr/idea/download/?section=windows



IntelliJ 下载

- IntelliJ
  - https://www.jetbrains.com/ko-kr/idea/download/?section=windows



IntelliJ Installation

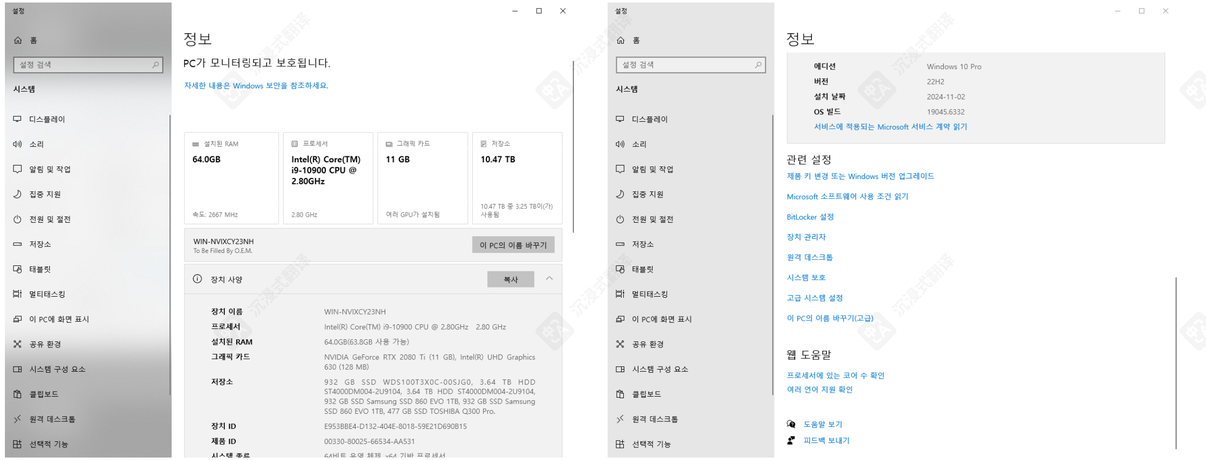
- IntelliJ
  - https://www.jetbrains.com/ko-kr/idea/download/?section=windows

IntelliJ 安装

- IntelliJ
  - https://www.jetbrains.com/ko-kr/idea/download/?section=windows

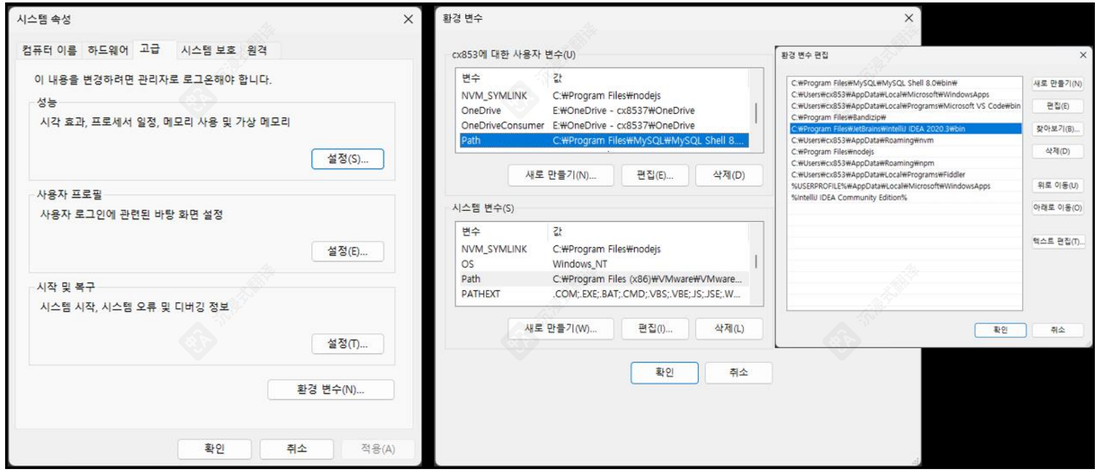
Environment Variables

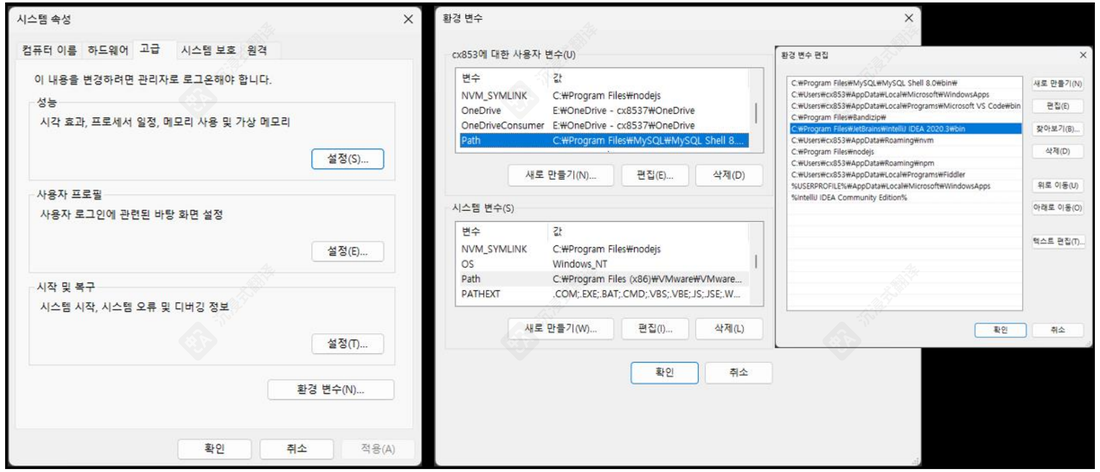
环境变量



Environment Variables

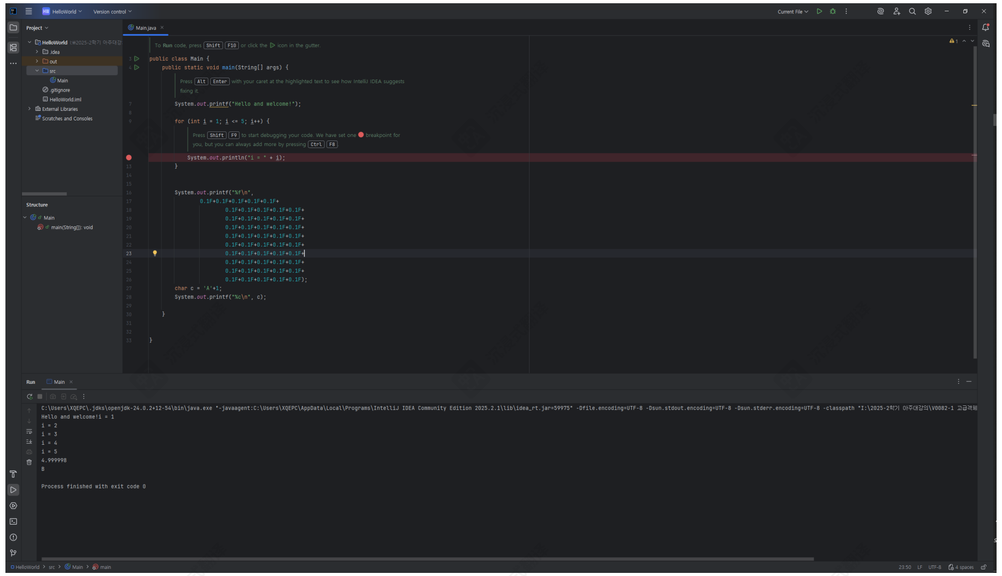
环境变量

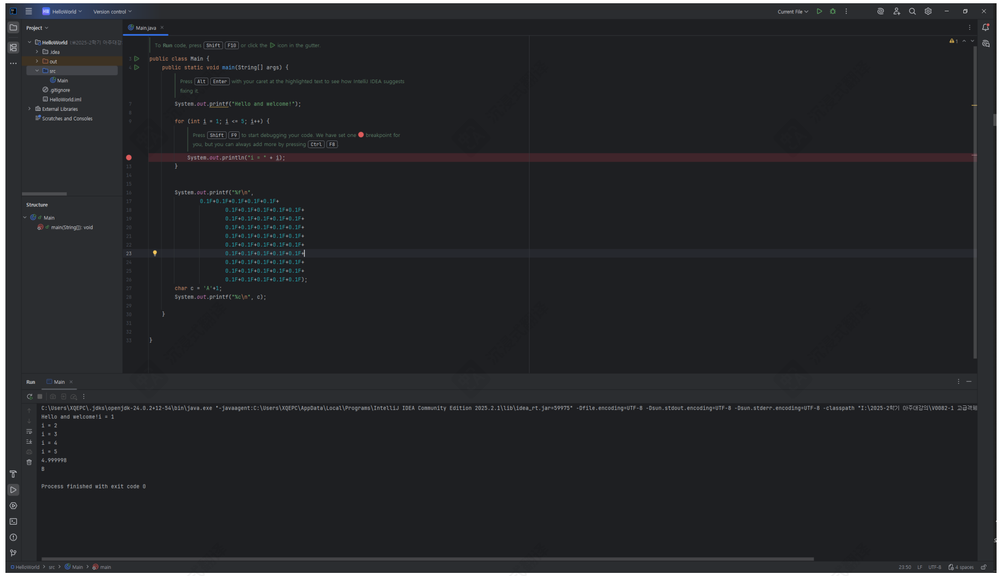




Environment Variables

环境变量





Useful Shortcut

- Build - Ctrl+F9
- Run - Shift+F10, Ctrl+F5
- Debug mode run - Shift+F9
- Break point – Ctrl+F8
- Force stop debugging – Ctrl+F2
- Relaunch – F9
- Step over – F8
- Step into – F7

实用快捷键

- 构建 - Ctrl+F9
- 运行 - Shift+F10, Ctrl+F5
- 调试模式运行 - Shift+F9
- 断点 - Ctrl+F8
- 强制停止调试 - Ctrl+F2
- 重新启动 – F9
- 单步跳过 – F8
- 单步进入 – F7

JAVA – Hello World

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

JAVA – 你好，世界

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

02

Buildtime and Runtime

02

构建时和运行时

아주대학교
AJOU UNIVERSITY

Buildtime and Runtime

- Build-time (including compile-time)
  - **.java** source code is translated into **.class** (Java byte code)
  - When using external libraries, only **symbolic references** exist; there is no concept of static build.
  - External **library** (class file) linking is handled by the JVM.

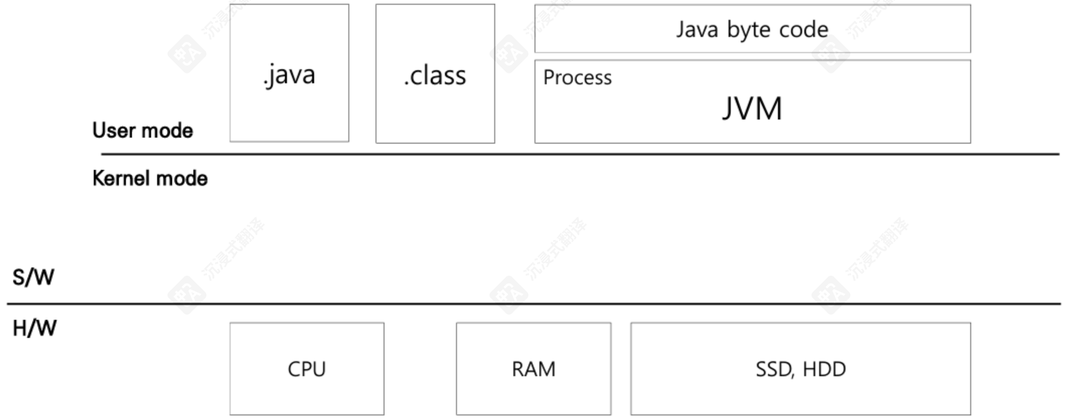
- Runtime (including link-time)
  - The JVM's class loader procedurally loads bytecode and loads it into memory.
  - Additional class files are also loaded and linked.

构建时和运行时

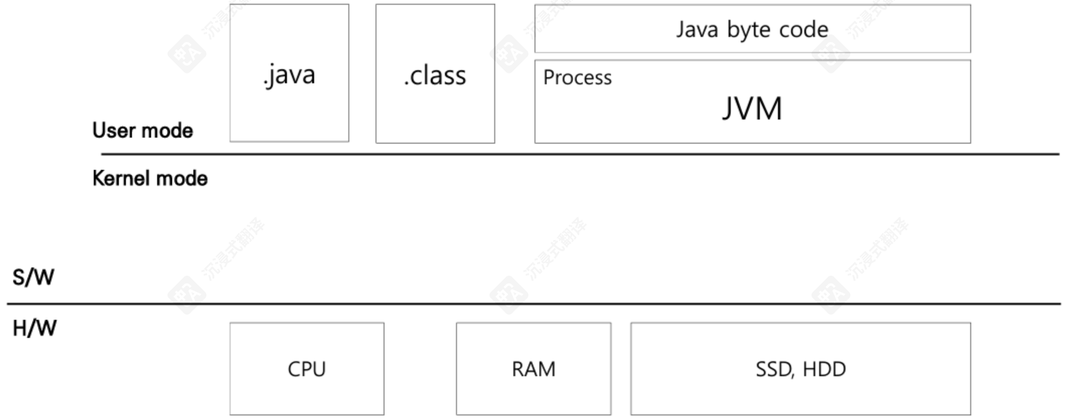
- 构建时（包括编译时）
  - .java 源代码被翻译成 .class（Java 字节码）
  - 在使用外部库时，仅存在符号引用；没有静态构建的概念
  - 外部库（类文件）的链接由 JVM 处理

- 运行时（包括链接时）
  - JVM 的类加载器按顺序加载字节码并将其加载到内存中
  - 其他类文件也会被加载和链接。

Compile and Link Time



编译和链接时间



03

JAVA vs. C/C++

03

JAVA vs. C/C++

아주대학교
AJOU UNIVERSITY

Java vs. C/C++

- Java
  - .class file
  - Reference .class file
  - Package (module level)
  - JVM class loader
  - Reference
  - Stack, Heap
- C/C++
  - .obj and .exe (PE files)
  - .lib, .dll (PE files)
  - Namespace (code level)
  - Linker
  - Pointers, references
  - Stack, Heap

Java 与 C/C++

- Java
  - .class 文件
  - 引用 .class 文件
  - 包（模块级别）
  - JVM 类加载器
  - 引用
  - 堆栈，堆
- C/C++
  - .obj 和 .exe (PE 文件)
  - .lib, .dll (PE 文件)
  - 命名空间 (代码级别)
  - 链接器
  - 指针，引用
  - 栈，堆

Java vs. C/C++

- There is no **default copy constructor**.
- Java's **reference structure** is nearly identical to that of C++, and the same **deep copy issue** persists.
- In Java, string constants are **String class objects**.
- The import statement is similar to the **#include** preprocessor (except that Java doesn't have a **preprocessor**).
- Java has no concept of **global variables**.
- Accessing static members from a static method doesn't require member access operations.
- The scope assignment operator (::) and member access operator (.) are used together.

Java 与 C/C++

- **没有默认的拷贝构造函数。**
- Java的引用结构几乎与C++相同，并且仍然存在相同的深拷贝问题。
- 在Java中，字符串常量是String类对象。
- 导入语句类似于预处理器的#include（除了Java没有预处理器的区别）。
- Java没有全局变量的概念。
- 从静态方法中访问静态成员不需要成员访问操作。
- 作用域赋值运算符（::）和成员访问运算符（.）一起使用。

Java vs. C/C++

- In C/C++, declarations and definitions are clearly separated into **.h** and **.cpp** files, respectively, but Java does not.
- Class names and file names must be case-sensitive.
- **Fields** instead of member data.
- If a field is a reference type, it can be initialized using the new operation in the class declaration code.

Java 与 C/C++

- 在 C/C++ 中，声明和定义分别清晰地分离到 .h 和 .cpp 文件中，但 Java 不是这样。
- 类名和文件名必须区分大小写。
- 字段而不是成员数据。
- 如果一个字段是引用类型，它可以在类声明代码中使用 new 操作进行初始化。

Quiz

- What are the biggest differences between the JDK and JRE, which are required to build a Java development environment?
  A. Development vs. Execution
  B. Paid vs. Free
  C. Windows vs. Mac
  D. Text vs. Graphical Interface

Quiz

- JDK 和 JRE 之间最大的区别是什么，这些区别对于构建 Java 开发环境是必需的?
  A. 开发与执行
  B. 付费与免费
  C. Windows 与 Mac
  D. 文本与图形界面

Quiz

- What are the core components that directly execute Java bytecode (.class files)?
  A. JDK
  B. JRE
  C. JVM
  D. Java Compiler

Quiz

- Java字节码的直接执行核心组件是什么(.class文件)?
  A. JDK
  B. JRE
  C. JVM
  D. Java编译器

Quiz

- What are the main names for the steps involved in converting source code to bytecode in Java?
  A. Runtime
  B. Link time
  C. Build time
  D. Execution time

Quiz

- 将Java源代码转换为字节码涉及的主要步骤有哪些名称?

  A. 运行时
  B. 链接时
  C. 构建时
  D. 执行时

Quiz

- What are the main activities involved in the Java runtime?
  A. Compiling and building
  B. Integrating external modules and executing code
  C. Debugging settings
  D. Generating source files

Quiz

- Java 运行时涉及哪些主要活动?
  A. 编译和构建
  B. 集成外部模块和执行代码
  C. 调试设置
  D. 生成源文件

Quiz

- What are the primary roles of an IDE (Integrated Development Environment) like IntelliJ IDEA?
  A. Operating system management
  B. Hardware performance optimization
  C. Integrating and facilitating the development process
  D. Website design

Quiz

- IDE（集成开发环境）的主要角色是什么？
  环境) 类似 IntelliJ IDEA?
  操作系统管理
  B. 硬件性能优化
  C. 集成并促进开发流程
  D. 网站设计

04

JAVA Datatype

04

JAVA 数据类型

아주대학교 AJOU UNIVERSITY

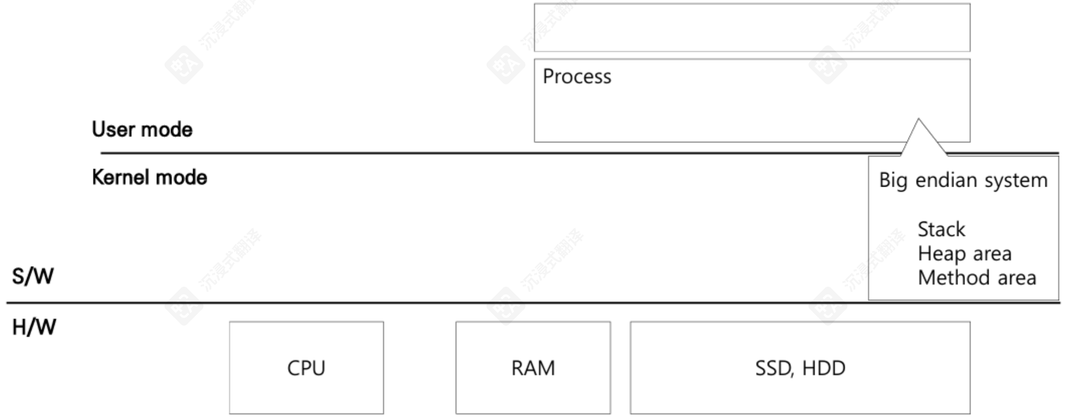
Features of JAVA

- OOP running on the JVM
  - Structurally eliminates memory management and responsibility issues, the most significant features of C/C++
  - No dependency on the OS (or platform)
  - Both compiler and interpreter features (hybrid)
  - Syntactically very similar to C/C++

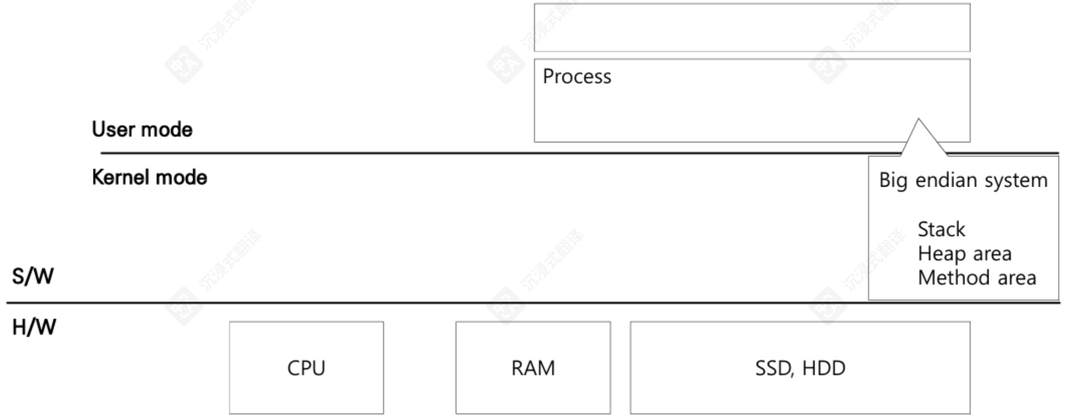
JAVA 的特性

- JVM 上的面向对象编程
  - 结构上消除了内存管理和责任问题，这是 C/C++ 最显著的特征
  - 不依赖于操作系统（或平台）
  - 编译器和解释器功能兼具（混合）
  - 语法与 C/C++ 非常相似

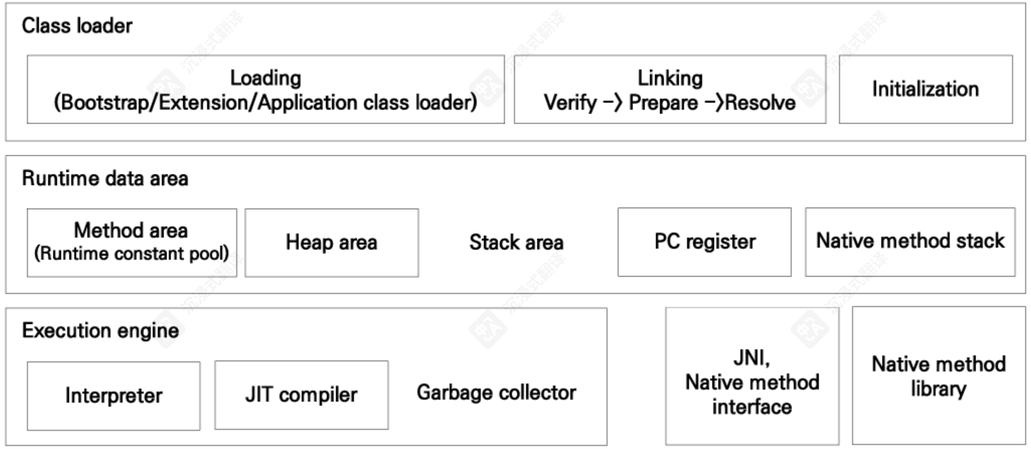
User mode process JVM



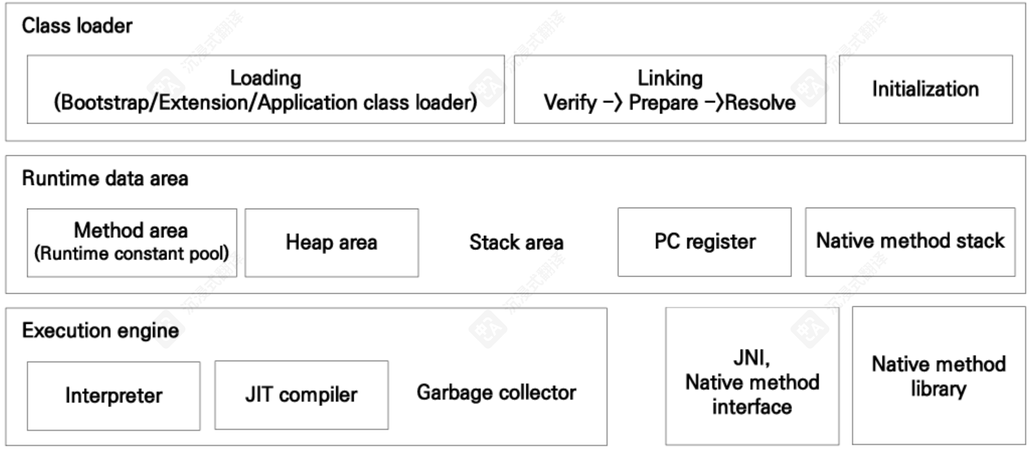
用户模式进程JVM



JVM Components



JVM 组件



JIT Compiler

- JIT (Just In Time)
  - Translates Java bytecode into actual machine code.
  - Optimizes code when the JVM detects inefficient code, such as repetitive code.
  - Collects execution history and applies it primarily to frequently used code.
  - **The longer a program runs, the more advantageous the performance improvement.**

JIT编译器

- JIT (Just In Time)
  - 将Java字节码翻译成实际的机器码。
  - 当JVM检测到低效代码时（例如重复代码）会优化代码。
  - 收集执行历史，主要应用于经常使用的代码。
  - **程序运行时间越长，性能提升的优势就越大。**

Hardware-level Data Types

- Integer
  - Representation range determined by number of bits
  - Subtraction implemented using **two's complement** addition
- IEEE 754-based real numbers
  - Single-precision (float)
  - Double-precision (double)

硬件级数据类型

- 整数
  - 表示范围由位数决定
  - 减法通过补码加法实现
- 基于IEEE 754的实数
  - 单精度（float）
  - 双精度（double）

Real Number Expression Range

实数表达式范围

float

Sign	8bit exponent(e)	23bit mantissa(m)

double

Sign	11bit exponent(e)	52bit mantissa(m)

float

Sign	8bit exponent(e)	23bit mantissa(m)

double

Sign	11bit exponent(e)	52bit mantissa(m)

Real Number

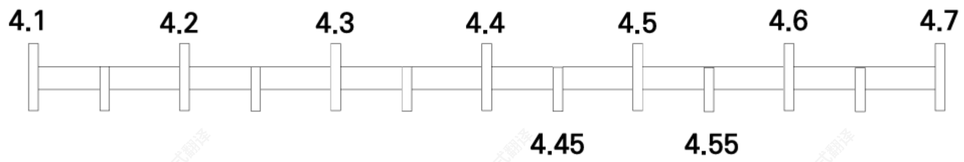
- IEEE(Institute of Electrical and Electronics) Standard Usage
  - IEEE 754
- Format for displaying decimal information
- Floating-point representation
- Because there are countless real numbers between two integers, a certain level of error (**floating-point error**) is **tolerated**.

实数

- IEEE(Institute of Electrical and Electronics) 标准使用
  - IEEE 754
- 显示十进制信息的格式
- 浮点数表示
- 因为两个整数之间存在无数个实数，所以容忍一定程度的误差（浮点数误差）。

Real Number

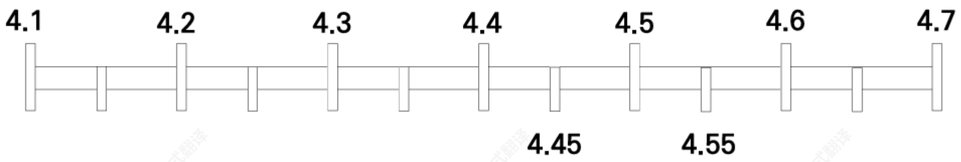
- Interview Questions
  - **Can we say that 4.999998 is the same as 5.0?**




```
System.out.printf("%f\n", 4.9999998F);
```

实数

- 面试问题
  - **我们可以认为4.999998和5.0是相同的吗?**




```
System.out.printf("%f\n", 4.9999998F);
```

Floating Point Error

浮点数错误

```java
public class Main {
    public static void main(String[] args) {
        System.out.printf("%f\n",
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F);
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        System.out.printf("%f\n",
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F +
                0.1F + 0.1F + 0.1F + 0.1F + 0.1F);
    }
}
```

JAVA Floating Point Type Range

JAVA 浮点数类型范围

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("float MIN: " + Float.MIN_VALUE);
        System.out.println("float MAX: " + Float.MAX_VALUE);
        System.out.println("double MIN: " + Double.MIN_VALUE);
        System.out.println("double MAX: " + Double.MAX_VALUE);
    }
}
```


```java
public class Main {
    public static void main(String[] args) {
        System.out.println("float MIN: " + Float.MIN_VALUE);
        System.out.println("float MAX: " + Float.MAX_VALUE);
        System.out.println("double MIN: " + Double.MIN_VALUE);
        System.out.println("double MAX: " + Double.MAX_VALUE);
    }
}
```

Actual Accident 实际事故

朝鮮日報

오피니언 정치 사회 국제 건강 재테크 스포츠 문화·연예 쇼핑의맛

사회 > 신문은 선생님

나이스, 프로그램에 문제… 쓰레기값 (컴퓨터 연산과정서 드물게 나오는 엉뚱한 값) 처리 누락

개발 맡은 삼성SDS 프로그래머 실수로 판명
고교 내신 동점자 처리 문제
원래 소수점 16자리까지 표시
일부선 평가와 다른 숫자 나와

김연주 기자
입력 2011.07.25. 03:02

가

https://www.chosun.com/site/data/html_dir/2011/07/25/2011072500100.html

朝鮮日報

오피니언 정치 사회 국제 건강 재테크 스포츠 문화·연예 쇼핑의맛

사회 > 신문은 선생님

나이스, 프로그램에 문제… 쓰레기값 (컴퓨터 연산과정서 드물게 나오는 엉뚱한 값) 처리 누락

개발 맡은 삼성SDS 프로그래머 실수로 판명
고교 내신 동점자 처리 문제
원래 소수점 16자리까지 표시
일부선 평가와 다른 숫자 나와

김연주 기자
입력 2011.07.25. 03:02

가

https://www.chosun.com/site/data/html_dir/2011/07/25/2011072500100.html

Java Primitive Types

- Integers
  - Byte, short, char, int, long
- Real Numbers
  - Float, double
- Logic
  - Boolean(true, false)

Java 基本类型

- 整数
  - 字节, 短整型, 字符型, 整型, 长整型
- 实数
  - Float , double
- 逻辑
  - Boolean(true, false)

Java Primitive Types

Type	Size (in bits)	Range
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2^{31} to $2^{31}-1$
long	64	-2^{63} to $2^{63}-1$
float	32	1.4e-045 to 3.4e+038
double	64	4.9e-324 to 1.8e+308
char	16	0 to 65,535
boolean	1	true or false

Java 基本类型

Type	Size (in bits)	Range
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2^{31} to $2^{31}-1$
long	64	-2^{63} to $2^{63}-1$
float	32	1.4e-045 to 3.4e+038
double	64	4.9e-324 to 1.8e+308
char	16	0 to 65,535
boolean	1	true or false

https://www.startertutorials.com

Non-Primitive Types

- Non-primitive or Derived type
- What is referred to as an object in Java
- String
  - Class for handling strings
- Class
  - Array, List, Queue, Stack
  - Interface

非原始类型

- 非原始或派生类型
- Java中所谓的对象是什么
- String
  - 处理字符串的类
- 类
  - 数组、列表、队列、栈
  - 接口

Other Formats

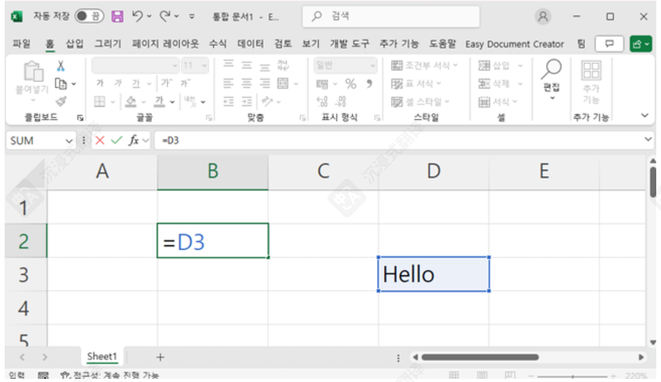
- **void** type
- Function type

其他格式

- **void类型**
- 函数类型

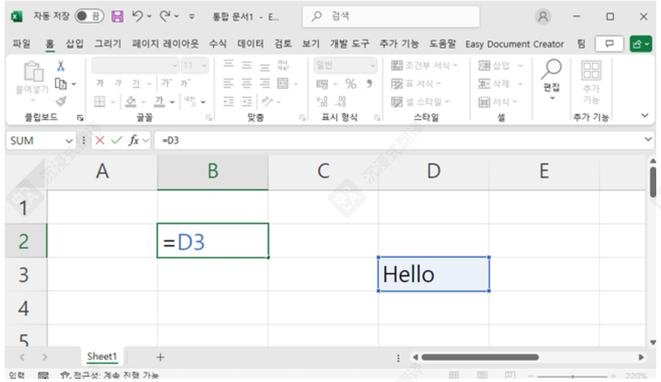
References in JAVA

- Objects and Data Types
- Primitive Data Types
- Reference Data Types and User-Defined Types



JAVA中的引用

- 对象和数据类型
- 基本数据类型
- 引用数据类型和用户定义类型



Object, Class, Instance

- An object is a set of data (variables) required for a specific purpose, grouped together to form a single entity.
- A class is a syntax for describing objects and can be a new data type.
- An instance is a variable for a specific data type.
  - ex) int a;
  - ex) String str_hello = new String(“Hello”)

对象，类，实例

- 一个对象是一组用于特定目的的数据（变量），它们被组合在一起形成一个单一实体。
- 一个类是描述对象的语法，并且可以是一种新的数据类型。
- 一个实例是特定数据类型的变量。
  - ex) int a;
  - 示例) 字符串str_hello = new String(“Hello”)

Characters and Strings

- A single English or Korean character is defined as a "character.“
- A “String" is a series of characters arranged in a contiguous array.

字符和字符串

- 一个英文字符或韩文字符被定义为“字符。 “
- “字符串”是一系列连续排列的字符。

Object, Class, Instance

- The char type is a data type for storing a single character (Unicode).
  - char ch = ‘A’;
- The char type belongs to the integer type.
  - ‘A’+1;
- A string is an array of multiple char data.
  - char[] hello = {‘H’, ‘e’, ‘l’, ‘l’, ‘o’};

对象，类，实例

- 字符类型是一种用于存储单个字符（Unicode）的数据类型。
  - char ch = ‘A’;
- 字符类型属于整数类型。
  - ‘A’+1;
- 字符串是由多个字符数据组成的数组。
  - char[] hello = {‘H’, ‘e’, ‘l’, ‘l’, ‘o’};

Unicode and Encoding Rules

- Strings in C are divided into MBCS (Multi-byte Character Sets) and Unicode strings.
- Unicode uses two bytes for each character, including both Korean and English.
- Java natively supports both the UTF-16 and Modified UTF-8 systems.

Unicode和编码规则

- C语言中的字符串分为MBCS（多字节字符集）和Unicode字符串。
- Unicode每个字符使用两个字节，包括韩文和英文。
- Java原生支持UTF-16和修改后的UTF-8系统。

Character Output

角色输出

```java
public class Main {
    public static void main(String[] args) {
        System.out.println('A');
        System.out.println((char)(65 + 1));
        System.out.println((char)('A' + 2));
    }
}
```


```java
public class Main {
    public static void main(String[] args) {
        System.out.println('A');
        System.out.println((char)(65 + 1));
        System.out.println((char)('A' + 2));
    }
}
```

Character Output

角色输出

```java
public class Main {
    public static void main(String[] args) {
        char ch = 'A';
        System.out.println(ch);
        char[] hello = {'H','e', 'l', 'l', 'o'};
        System.out.println(hello);
        String strHello = "Hello";
        System.out.println(strHello + " World");
    }
}
```


```java
public class Main {
    public static void main(String[] args) {
        char ch = 'A';
        System.out.println(ch);
        char[] hello = {'H','e', 'l', 'l', 'o'};
        System.out.println(hello);
        String strHello = "Hello";
        System.out.println(strHello + " World");
    }
}
```

Quiz

- What is the primary role of the Java Virtual Machine (JVM)?
  A. It directly translates Java source code into machine code.
  B. It provides a virtual environment for Java programs to run on various operating systems.
  C. It directly controls hardware to optimize performance.
  D. It helps developers manually manage memory allocation and deallocation.

Quiz

- Java 虚拟机（JVM）的主要作用是什么？
  A. 它直接将 Java 源代码翻译成机器代码。
  B. 它为 Java 程序提供虚拟环境，使其能够在各种操作系统上运行。
  C. 它直接控制硬件以优化性能。
  D. 它帮助开发者手动管理内存分配和释放。

Quiz

- What automatically manages (reclaims) unused memory in Java?
  A. Developer
  B. Operating System
  C. Garbage Collector
  D. Class Loader

Quiz

- Java中自动管理（回收）未使用内存的是什么？
  A. 开发者
  B. 操作系统
  C. 垃圾回收器
  D. 类加载器

Quiz

- What are the main causes of errors in floating-point calculations on computers?
  - A. Limitations in CPU computational power
  - B. The difficulty of accurately representing decimals in binary
  - C. Caused by insufficient memory
  - D. Just-In-Time (JIT) compiler errors

Quiz

- 计算机在浮点计算中产生错误的主要原因是什么?

  A. CPU计算能力的局限性
  B. 二进制中准确表示小数的难度
  C. 由内存不足引起
  D. Just-In-Time (JIT) 编译器错误

Quiz

- In Java, where are objects created using the "new" keyword (i.e., reference-type objects) primarily stored in memory?
  A. Stack
  B. Heap
  C. Code
  D. Register

Quiz

在Java中，使用"new"关键字创建的对象在哪里（即，(reference-type objects) 主要存储在内存中?
栈
堆
C. 代码
D. 注册

Quiz

- In object-oriented programming, what concept is often likened to a "blueprint" or "framework," defining the structure and behavior of an object and acting like a new data type?
  A. Object
  B. Instance
  C. Class
  D. Reference

Quiz

- 在面向对象编程中，什么概念常被比喻为“蓝图”或“框架”，定义了对象的结构和行为，并像一种新的数据类型一样起作用?

  A. 对象
  B. 实例
  C. 类
  D. 引用

Q & A

问答

THANK YOU

谢谢